

Algorithmic Data Mining: Anomaly Detection for Credit-Card Fraud with Autoencoders

Assignment 2 Report

Luis Fernando Chavez Quevedo

Eneko Sabaté Iturgaiz

June 19, 2026

1 Introduction

Anomaly detection is an analytical method used to identify observations that differ significantly from the majority of the data. Thus, this is a classification problem with two classes: *normal* and *anomaly*. Common supervised methods to solve this problem include standard classifiers such as Bayesian classifiers, Support Vector Machines (SVMs), or Random Forests. However, approaches like these rely heavily on having sufficient samples of both classes, which is rarely the case in real-world data. Unsupervised methods such as AutoEncoders (AEs) can be used for this purpose by learning the latent structure of strictly *normal* data (since *anomalies* are not used to train the model, they will yield a significantly higher reconstruction error). This unsupervised perspective is robust and adaptive, eliminating the reliance on extensive *anomaly* samples.

This work presents an AE pipeline applied to the Credit Card Fraud Detection dataset¹, a highly unbalanced dataset where the positive class (fraud) accounts for only 0.172% of all transactions. Therefore, this is a perfect testbed to evaluate how AEs perform against strong unsupervised baselines.

2 Theoretical Framework

2.1 Autoencoders as undercomplete representation learning

An AE is a neural network trained to reconstruct its own input through a bottleneck. It is composed of an encoder $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^k$, which maps an input $\mathbf{x} \in \mathbb{R}^d$ to a latent code $\mathbf{z} = f_\theta(\mathbf{x})$, and a decoder $g_\phi : \mathbb{R}^k \rightarrow \mathbb{R}^d$, which maps the code back to a reconstruction $\hat{\mathbf{x}} = g_\phi(\mathbf{z})$. The AE is *undercomplete* when $k < d$: the narrow code forces the network to discard redundancy and retain only the dominant structure of the data, rather than copying its input. This constraint is essential, since an AE with enough capacity (a wide code or a sufficiently flexible decoder) can approximate the identity map and learn nothing useful about the data distribution [Goodfellow et al., 2016, ch. 14].

The undercomplete AE is best understood as nonlinear representation learning. A *linear* AE trained with squared error spans the same subspace as Principal Component Analysis (PCA), i.e. the one generated by the leading principal components [Goodfellow et al., 2016, ch. 14]; the low-rank structure this exposes is the same one made explicit by the singular value decomposition and related matrix factorizations [Golub and Van Loan, 2013, ch. 2]. A nonlinear AE generalizes this idea to a curved low-dimensional manifold, which is what we exploit for anomaly detection. In our pipeline $d = 31$ and $k = 16$ (see Section 3.5).

2.2 Reconstruction error as an anomaly score

We train the AE on legitimate transactions only. The network therefore learns the manifold of normal behaviour: it reconstructs in-distribution (legitimate) points accurately, but, never having seen fraud, it reconstructs anomalous points poorly. The per-sample reconstruction error is then used directly as an anomaly score,

$$s(\mathbf{x}) = \frac{1}{d} \|\mathbf{x} - g_\phi(f_\theta(\mathbf{x}))\|_2^2, \quad (1)$$

so that the larger the error, the more the transaction departs from the learned notion of normality. This is the one-class (semi-supervised) anomaly-detection setting: a binary decision is obtained by thresholding $s(\mathbf{x})$, with the threshold fixed on a validation set and never on test (Section 3.7). The convention that a *higher score means more anomalous* is shared by every detector in this work, which lets us evaluate all of them with a single, uniform procedure.

¹<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

2.3 Training: backpropagation, loss and optimization

The parameters θ, ϕ are fit by minimizing the mean reconstruction loss over the N legitimate training transactions,

$$\mathcal{L}(\theta, \phi) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - g_\phi(f_\theta(\mathbf{x}_i))\|_2^2, \quad (2)$$

the Mean Squared Error (MSE) objective. Its gradients with respect to all weights are obtained by backpropagation, the reverse-mode application of the chain rule through the network [Goodfellow et al., 2016, ch. 6][Nielsen, 2015, chs. 1–2], and the parameters are updated with Adam, an adaptive first-order stochastic optimizer [Goodfellow et al., 2016, ch. 8].

A sufficiently flexible AE can overfit and, in the limit, reconstruct everything well enough to blur the normal/anomalous distinction; the capacity–generalization trade-off that governs this is standard [Goodfellow et al., 2016, ch. 5]. We control it in two ways: the undercomplete bottleneck limits capacity by construction, and we apply early stopping, monitoring the reconstruction loss on a held-out set and restoring the checkpoint with the lowest validation loss. Early stopping is a well-understood regularizer [Goodfellow et al., 2016, ch. 7]. Crucially, the validation set used for early stopping contains *only legitimate* transactions, so model selection never observes fraud and the one-class assumption is preserved end to end.

2.4 Baseline detectors

We compare the AE against three unsupervised baselines. To keep the comparison fair, each baseline is fit on the same legitimate-only training data and exposes the same “higher score = more anomalous” convention as the AE; all three use their `scikit-learn` or `scipy` implementations [Pedregosa et al., 2011].

Isolation Forest (Isolation Forest (IF)). An ensemble of random binary trees, each built by repeatedly selecting a feature and a random split value. Because anomalies are few and different, they tend to be separated from the rest with very few splits, i.e. closer to the root. The anomaly score is a monotonically decreasing function of the expected path length required to isolate a point, so short paths yield high scores [Liu et al., 2008].

One-Class Support Vector Machine (OC-SVM). A kernel method that maps inputs into a reproducing-kernel feature space and finds the maximum-margin hyperplane separating the data from the origin. The parameter $\nu \in (0, 1]$ upper-bounds the fraction of training points allowed on the outlier side and lower-bounds the fraction of support vectors [Schölkopf et al., 2001]; we use the RBF kernel, and the underlying margin and kernel machinery follows the standard treatment [Schölkopf and Smola, 2002]. The signed distance to the boundary provides the score.

Gaussian density. A single multivariate Gaussian $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ is fit to the legitimate transactions by maximum likelihood (so $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ are the sample mean and covariance), and each point is scored by its negative log-likelihood $-\log p(\mathbf{x})$, so that low-density points score as anomalies. The Gaussian distribution itself is standard course material [Goodfellow et al., 2016, ch. 3]; its *use as a density-based anomaly detector*, however, is external to the course and we flag it as such. A small ridge is added to the diagonal of $\boldsymbol{\Sigma}$ for numerical stability (Section 3.6).

2.5 Evaluation under extreme imbalance: PR-AUC vs ROC-AUC

All four detectors output a continuous anomaly score, so they are compared with threshold-free ranking metrics. At a given threshold, with TP, FP, FN and TN the usual confusion-matrix counts, precision = $\text{TP}/(\text{TP} + \text{FP})$ and recall = $\text{TP}/(\text{TP} + \text{FN})$. Sweeping the threshold traces the precision–recall curve, summarised by the Average Precision (AP) (the Precision–Recall Area Under the Curve (PR-AUC)). The ROC curve instead traces the true-positive rate against the false-positive rate $\text{FPR} = \text{FP}/(\text{FP} + \text{TN})$, with area Receiver Operating Characteristic Area Under the Curve (ROC-AUC).

Under the extreme imbalance of this dataset (a fraud prevalence $\pi \approx 0.0017$), the two metrics tell very different stories. The negative class dominates the FPR denominator ($\text{FP} + \text{TN}$), so even thousands of false positives move the FPR only marginally and ROC-AUC stays optimistically high. Precision, by contrast, places FP next to TP in its denominator and is immediately sensitive to false alarms raised on the majority class, which makes PR-AUC the proper primary metric here. Davis and Goadrich [2006] formalize this relationship: a curve dominates in ROC space if and only if it dominates in PR space, yet the two spaces convey very different visual and area impressions, and PR curves expose differences on the minority class that ROC curves hide. A further consequence we rely on is that the no-skill baseline in PR space is the prevalence π (a random classifier

attains precision π at every recall), not the 0.5 of ROC space. We report both metrics but treat PR-AUC as primary throughout.

3 Implementation

3.1 Tooling and constraints

The core of this work revolves around the implementation of an unsupervised anomaly detection pipeline. While standard frameworks are utilized for data handling and baseline comparisons, the neural network architecture and evaluation logic are specifically tailored for this task. Below we justify each of the third-party libraries used in our pipeline:

- **torch** (PyTorch) for building, training, and evaluating the neural network AE architecture, as well as handling tensor operations and hardware acceleration.
- **scikit-learn** (`IsolationForest`, `OneClassSVM`, `StandardScaler`) only for the baseline anomaly detection models used in the comparison and the stateful feature scaling.
- **numpy** and **pandas** for all array operations, dataset loading, and mathematical transformations.
- **sklearn.metrics** (`precision_recall_curve`, `average_precision_score`, `roc_auc_score`) for calculating the performance baselines and extracting the continuous anomaly evaluation bounds.
- **matplotlib** for generating all the evaluation figures and exploratory data analysis plots.

No alternative, pre-packaged implementations of Autoencoders are imported: the neural network architecture, the custom training loops, and the rigorous stratified bootstrap evaluation strategies are all coded directly for this pipeline.

3.2 Reproducibility

To ensure the complete reproducibility of this study, all stochastic processes and hyperparameters are strictly controlled. A single file (`config.py`) is utilized to keep all the configuration settings centralized. Within this file, a fixed global random seed is set, alongside all structural model parameters (*e.g.*, latent dimensions, batch sizes, and learning rates). This guarantees that the neural network’s weight initialization, data shuffling, and bootstrap resampling yield purely deterministic results across multiple executions.

Furthermore, a persistent caching mechanism is implemented to save the intermediate anomaly scores and fit times from the latest experimental run into localized `.npz` arrays. This approach not only mitigates the high computational cost of retraining complex models (such as the One-Class SVM, which requires extensive fit times), but also allows all evaluation metrics, confidence intervals, and plots to be perfectly regenerated directly from the cached states without having to retrain the underlying models.

3.3 Dataset and preprocessing

The experiments are conducted on the Credit Card Fraud Detection dataset, which contains 284,807 European credit card transactions spanning a 48-hour period. To prepare the data for the AE and baselines, the following preprocessing pipeline is applied:

- **Duplicate removal:** The raw dataset contains exactly 1,081 fully duplicated rows, of which only 19 are fraud (the remaining 1,062 are legitimate). They are removed *before* splitting so that identical records cannot leak across the train/validation/test partitions; this discards only 19 of the 492 frauds ($\approx 3.9\%$), leaving 473, so the fraud signal is essentially preserved.
- **Cyclical time encoding:** The raw *Time* feature (representing seconds elapsed since the first transaction) is strictly linear, which fails to capture day/night patterns. To address this, the 24 hours of the day are mapped onto a unit circle using sine and cosine transformations. This ensures that temporally adjacent hours (*e.g.*, hour 23 and hour 0) are mathematically represented as close in the feature space, after which the original linear variable is discarded.
- **Amount normalization:** The transaction *Amount* feature exhibits extreme right-skewness (raw skewness ≈ 16.98). It is first compressed using a logarithmic transformation ($\log(1 + x)$) to normalize its distribution (adjusted skewness ≈ 0.16). Subsequently, a standard scaler is applied to force the feature to a zero mean and unit variance. Crucially, to prevent structural data leakage, this scaler is fitted exclusively on the training split and then blindly applied to transform the validation and test splits.

- **Class imbalance handling:** The dataset exhibits severe class imbalance, with fraudulent transactions accounting for merely 0.172% of the data. Because unsupervised anomaly detection models learn the latent manifold of strictly normal behavior, the positive class (fraud) is entirely excluded from the training phase. All positive samples are reserved strictly for the validation and test splits. The remaining 28 PCA components (V1 through V28) are passed through directly alongside the preprocessed *Time* (one sine and one cosine) and *Amount* features, resulting in a final input dimension of 31 features for the models.

3.4 Split strategies

To rigorously evaluate the models and ensure conclusions are not artifacts of a specific data partition, the pipeline implements a configurable `DataSplitter` interface. The dataset is partitioned into three sets: training (70%), validation (15%), and test (15%). The training set is strictly filtered to contain only normal transactions for model fitting, while the validation set (used for threshold tuning) and the test set (used for final evaluation) contain a mix of both classes. We implement two distinct splitting strategies:

- **Stratified Random Split:** The dataset is shuffled and partitioned randomly while strictly stratifying by the target class. This forces the validation and test sets to maintain the exact global fraud proportion ($\approx 0.17\%$). This strategy provides a mathematically controlled environment to test the models' baseline pattern-recognition capabilities without artificially inflating precision.
- **Temporal Split:** Transactions are sorted chronologically by the *Time* feature. The earliest 70% of transactions form the training set, followed by the next 15% for validation, and the final 15% for testing. This protocol mimics a realistic production environment—training on historical data to predict future fraud—and exposes the models to the natural non-stationarity and concept drift of real-world fraud tactics. In this split, the fraud rate in each block is allowed to fluctuate naturally rather than being forced to a fixed percentage.

3.5 Autoencoder architecture and training

The primary unsupervised model is a deep AE implemented using PyTorch. The network utilizes a symmetric, feed-forward architecture designed to compress the 31-dimensional input feature space into a dense bottleneck.

The encoder comprises an initial linear layer projecting the 31 inputs to 64 hidden units, followed by a ReLU activation, and a subsequent linear layer compressing the representation into a latent dimension of 16 units, also followed by a ReLU activation. The decoder mirrors this structure, expanding the 16-dimensional latent vector back to 64 hidden units (ReLU), and finally to the 31-dimensional output space. Crucially, the final output layer applies no non-linear activation function, as the standardized input features are unbounded.

The network is optimized to minimize the MSE between the input and its reconstruction, utilizing the Adam optimizer with a learning rate of 1×10^{-3} . Training is executed in mini-batches of 32 over a maximum of 100 epochs. To prevent overfitting, an early-stopping mechanism is employed: at the end of each epoch, the model is evaluated against a strictly legitimate subset of the validation data. The network's weights are checkpointed, and the state yielding the lowest validation reconstruction error is restored at the end of training. At inference time, MSE serves as the indicator for anomaly detection.

3.6 Baseline configuration

To benchmark the efficacy of the AE, its performance is compared against three established unsupervised baseline algorithms. Following the exact same paradigm as the neural network, all three baselines are fitted exclusively on the legitimate transactions within the training split. Once fitted, they are evaluated on the complete validation and test splits. Rather than outputting binary predictions directly, each model is configured to yield its continuous decision function, negated where necessary so that a higher score always indicates a higher likelihood of anomaly. This allows all models to be evaluated using the exact same threshold-tuning strategies. The baselines and their specific configurations are defined as follows:

- **Gaussian Density:** A multivariate normal distribution fitted to the normal training data. To ensure numerical stability and prevent singular covariance matrices, a minor regularization term (`reg_covar` = 1×10^{-6}) is added to the diagonal of the covariance matrix. The continuous anomaly score is extracted as the negative log-likelihood of the sample.
- **Isolation Forest:** An ensemble of isolation trees implemented via `scikit-learn`. The forest is grown using `n_estimators` = 100 (the scikit-learn default). The `contamination` parameter is set to "auto"; because the model's internal binary prediction is discarded in favor of the continuous anomaly score (the negated `score_samples`), the contamination prior does not affect the final ranking.

- **One-Class SVM:** A support vector margin estimator implemented via `scikit-learn`. The model utilizes a Radial Basis Function kernel type (`kernel="rbf"`) with a scale-adjusted gamma coefficient (`gamma="scale"`). The margin hyperparameter is set to the default `nu = 0.5`, which bounds the fraction of support vectors and training errors. The continuous anomaly score is extracted from the negated `decision_function`.

3.7 Evaluation protocol

Given the extreme class imbalance of the dataset ($\approx 0.17\%$ fraud), standard classification accuracy and the Receiver Operating Characteristic (ROC) curve can present an overly optimistic view of model performance. Consequently, the primary evaluation metric for the continuous anomaly scores is the PR-AUC. The no-skill baseline for PR-AUC is explicitly defined as the prevalence of the positive class in the test set, providing a strict benchmark for improvement.

To quantify the uncertainty of the models' performance in a rigorous way without retraining any model, the pipeline employs a bootstrap over the test set. For each model, the test set is resampled with replacement 1,000 times and the metric is recomputed on every resample (resamples that happen to contain a single class are discarded, since PR-AUC/ROC-AUC are then undefined). This generates 95% Confidence Intervals (CIs) for both the PR-AUC and ROC-AUC estimates. Bootstrapping the test set, rather than retraining under multiple seeds, is what makes the uncertainty quantification affordable given the One-Class SVM's long fit time.

Finally, because real-world fraud detection systems require binary decisions, the models are evaluated under three distinct thresholds depending on user preference:

1. **F1-Maximizing:** Balances false positives and false negatives by maximizing the harmonic mean of precision and recall.
2. **High Recall:** Calibrated to prioritize catching fraud, identifying the threshold that guarantees a minimum recall of 0.75.
3. **High Precision:** Calibrated to minimize false alarms, identifying the threshold that guarantees a minimum precision of 0.75.

Note that these specific thresholds are calculated strictly using the anomaly score distributions of the *validation* set. They are subsequently applied blindly to the *test* set, where a standard classification report is generated alongside bootstrapped 95% CIs for the fraud class's final Precision and Recall.

4 Results

4.1 Autoencoder performance

The AE demonstrates strong anomaly detection capabilities when evaluated on test data. Given the extreme class imbalance (a no-skill prevalence baseline of merely 0.0017), the model achieved a notable Precision-Recall Area Under the Curve (PR-AUC) of 0.6283 [95% CI: 0.5131, 0.7384]. The ROC-AUC score was highly optimistic at 0.9524 [95% CI: 0.9087, 0.9824] due to the aforementioned class imbalance, visually confirming the model's overall ability to push fraudulent transactions toward higher reconstruction errors. The evaluation curves for the AE are depicted in Figure 1.

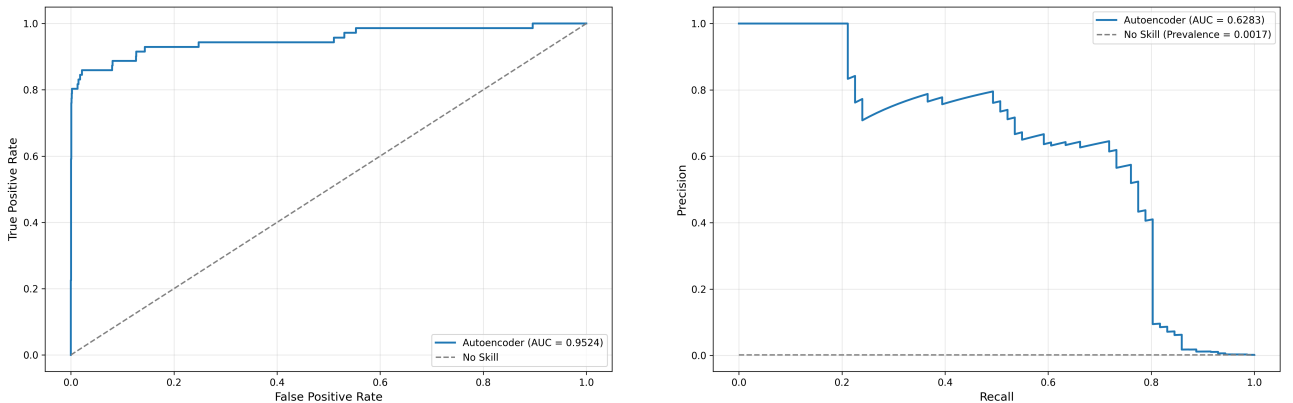


Figure 1: ROC and Precision-Recall evaluation curves for the Autoencoder. The dashed lines represent the no-skill baseline (random guessing).

Evaluating the AE under distinct thresholds demonstrates the classifier’s practicality under different circumstances. Table 1 details the model’s exact precision and recall CIs across three distinct strategies. When calibrated to maximize the F1-score, the model achieves a test precision of 0.5400 and a recall of 0.7606. This indicates that roughly 54% of the transactions flagged for manual review would indeed be fraudulent, while successfully catching 76% of all fraud. If the business priority is shifted toward catching more fraud (High Recall strategy), the threshold can be relaxed to catch about 80% of fraudulent transactions, though this degrades precision to roughly 0.24. Conversely, an extremely strict threshold (High Precision strategy) achieves a perfect precision of 1.00 (zero false alarms), but captures only roughly 20% of the total fraud.

Table 1: Operational threshold strategies for the Autoencoder. Metrics denote point estimates alongside 95% bootstrapped confidence intervals for the fraud class.

Threshold Strategy	Precision [95% CI]	Recall [95% CI]
F1-Maximizing	0.5400 [0.4395, 0.6381]	0.7606 [0.6582, 0.8572]
High Recall (Catch more fraud)	0.2415 [0.1868, 0.2992]	0.8028 [0.7058, 0.8889]
High Precision (Fewer false alarms)	1.0000 [1.0000, 1.0000]	0.1972 [0.1111, 0.2879]

4.2 Model comparison

To benchmark the AE, its performance was compared against the three unsupervised baselines. Table 2 details the continuous metrics and the computational time required to fit each model.

Table 2: Model comparison based on continuous metrics and fit times. Intervals represent 95% bootstrapped confidence limits.

Model	Fit Time (s)	PR-AUC	ROC-AUC
Autoencoder	1214.5	0.6283 [0.5131, 0.7384]	0.9524 [0.9087, 0.9824]
Gaussian Density	0.1	0.4993 [0.3813, 0.6176]	0.9608 [0.9337, 0.9834]
One-Class SVM	2747.9	0.2323 [0.1457, 0.3338]	0.9591 [0.9341, 0.9806]
Isolation Forest	0.5	0.1198 [0.0763, 0.1985]	0.9578 [0.9315, 0.9801]

As expected, the ROC-AUC scores are very optimistic, with their CIs densely clustered between 0.95 and 0.96 for all models, failing to distinguish their true operational efficacy. However, the PR-AUC exposes significant variations in performance.

The AE attains the highest PR-AUC (0.6283), ahead of the Gaussian Density estimator (0.4993); because their confidence intervals overlap, the AE leads but not conclusively. Both the AE and the Gaussian clearly outperform the other two, whose intervals do not overlap theirs: the One-Class SVM (0.2323) and the Isolation Forest (0.1198). The One-Class SVM is also by far the most computationally demanding, taking roughly 46 minutes to fit, yet ranking only third. The Isolation Forest, though near-instant to fit, fails to separate the classes effectively, yielding the lowest PR-AUC. Notably, the Gaussian is essentially free to fit (0.1 s) while remaining the AE’s closest competitor, a strong argument for it as a cheap, robust baseline.

4.3 Split-strategy comparison

The results presented in Section 4.2 rely on a randomized partitioning of the dataset. While mathematically sound for measuring generalized pattern recognition, random splitting ignores the arrow of time, allowing models to implicitly learn from future behavioral distributions. To evaluate how these models would perform in a real-world, non-stationary environment, they were re-evaluated using a chronological split (training on past transactions, predicting on future ones). The performance decay across both splits is summarized in Table 3.

The chronological protocol reveals a critical insight regarding non-stationarity: *every* method degrades substantially under temporal validation, with PR-AUC roughly halving across the board (the AE drops from 0.6283 to 0.2381, the Gaussian from 0.4993 to 0.2323). This indicates that the statistical distribution of legitimate transactions shifted over the 48-hour period (concept drift), so models fitted on the past flag normal future transactions as anomalous. Crucially, ROC-AUC stays high (≈ 0.92 – 0.95) in both splits, masking this degradation entirely and reinforcing the choice of PR-AUC as the primary metric.

The AE and the Gaussian remain the top tier under drift, but are now statistically tied: the AE (0.2381) and the Gaussian (0.2323) have heavily overlapping confidence intervals, so neither can be declared best on the temporal split. Both stay clearly ahead of the One-Class SVM (0.1220) and the Isolation Forest, the

Table 3: Comparison of PR-AUC across split strategies. The temporal split exposes the models’ susceptibility to concept drift.

Model	Random Split PR-AUC	Temporal Split PR-AUC
Autoencoder	0.6283	0.2381
Gaussian Density	0.4993	0.2323
One-Class SVM	0.2323	0.1220
Isolation Forest	0.1198	0.0384

latter collapsing almost to the prevalence floor (0.0384) and proving the most drift-sensitive method. At its F1-maximizing operating point on strictly future fraud, the AE reaches a precision of 0.26 and a recall of 0.56.

4.4 Operation points

The three geometric strategies of Section 3.7 describe the precision/recall trade-off but do not indicate *which* point to deploy. We therefore add a business-cost criterion: an asymmetric cost matrix that assigns a cost C_{FN} to a missed fraud (false negative) and C_{FP} to a false alarm (false positive), and selects, *on validation*, the threshold that minimizes the expected cost

$$\text{Cost}(t) = C_{FN} \cdot \text{FN}(t) + C_{FP} \cdot \text{FP}(t), \quad (3)$$

which is then evaluated once on test. Each model selects its own threshold, since the score scales are not comparable across models. The natural reference is a system that flags nothing (all frauds missed), whose cost is $C_{FN} \cdot n_{\text{fraud}}$.

Table 4: Minimum-cost operating point at a moderate 2:1 ratio ($C_{FN} = 100$, $C_{FP} = 50$). Threshold chosen on validation; counts and cost reported on test (stratified-random split, 71 frauds).

Model	Test FP	Test FN	Test cost
Autoencoder	28	21	3500
Gaussian Density	85	20	6250
One-Class SVM	6	66	6900
Isolation Forest	0	71	7100
<i>(flag nothing)</i>	0	71	7100

At this moderate 2:1 ratio on the stratified split (Table 4), only the AE substantially beats taking no action (cost 3500 vs. 7100), roughly halving it; the Gaussian helps less (6250) and the boundary/tree methods barely (One-Class SVM, 6900) or not at all (Isolation Forest, which falls back to flagging nothing). On the *temporal* split the result is degenerate: at 2:1 every model’s cost-optimal action is to flag nothing (cost 5200), because under concept drift a false alarm priced at half a missed fraud is simply not worth raising for so rare a positive.

Because a single ratio is arbitrary, we sweep C_{FN}/C_{FP} (fixing $C_{FP} = 1$); Table 5 reports the resulting test cost. The sweep exposes a clear *break-even ratio*. On the stratified split the AE pays off from 2:1 and is the cheapest option up to 50:1. On the temporal split no detector beats no-action until a missed fraud is at least $5\times$ (Gaussian) or $10\times$ (AE) more costly than a false alarm, and $50\times$ for the One-Class SVM and Isolation Forest. At the extreme 1000:1 ratio the regime becomes recall-dominated (false positives almost free) and the ranking reshuffles, with Isolation Forest cheapest on the stratified split. The practical message is that whether to deploy a detector (and which one) is governed by the business cost ratio, not by PR-AUC alone.

5 Discussion

5.1 Does the simplest model win? Gaussian vs the autoencoder

Under the stratified split the AE attains the highest PR-AUC (0.6283), ahead of the Gaussian density estimator (0.4993). However, the two confidence intervals overlap substantially ($[0.5131, 0.7384]$ vs. $[0.3813, 0.6176]$), so the AE’s advantage, while consistent, is not statistically conclusive on a single run. The more striking observation is how competitive the Gaussian is: a single multivariate Gaussian fitted in 0.1 s rivals a neural network that takes roughly twenty minutes to train. A plausible explanation lies in the data itself: features V1–V28 are already PCA components, hence approximately decorrelated and individually near-Gaussian, which is close to ideal terrain for a single multivariate normal. The non-linear manifold learned by the AE adds only a modest

Table 5: Cost-ratio sensitivity: test cost per model as the ratio $C_{FN}:C_{FP}$ varies ($C_{FP} = 1$, threshold chosen on validation). A * marks a model whose cost-optimal action is to flag nothing (cost = the no-action baseline).

Model	2:1	5:1	10:1	50:1	100:1	1000:1
<i>Stratified-random split</i>						
Autoencoder	70	149	229	908	1608	12404
Gaussian Density	125	185	281	882	1582	14465
One-Class SVM	138	290	463	1565	2315	12559
Isolation Forest	142*	355*	634	1792	2492	10476
(flag nothing)	142	355	710	3550	7100	71000
<i>Temporal split</i>						
Autoencoder	104*	260*	312	823	1518	12534
Gaussian Density	104*	180	286	966	1694	12209
One-Class SVM	104*	260*	520*	1598	2391	13442
Isolation Forest	104*	260*	520*	1905	2982	13154
(flag nothing)	104	260	520	2600	5200	52000

edge when the legitimate data is already well described in a linear, Gaussian basis. The real reading is therefore not “the deep model wins” but “the AE earns a small, non-decisive edge over an almost-free baseline”, a useful corrective to the assumption that a more complex model must dominate.

5.2 Reconstruction/density vs boundary/tree methods

A robust hierarchy emerges that, unlike the AE-vs-Gaussian question, *is* statistically clear: the two methods that model the normal data directly (the AE through error reconstruction, and the Gaussian through density) outperform the boundary-based One-Class SVM, which in turn beats the tree-based IF, on PR-AUC in *both* splits, with non-overlapping confidence intervals separating this top tier from the rest. This ordering is interpretable. Fraud lies away from the structure of legitimate behaviour (off the manifold the AE learns to reconstruct, and in the low-density tails the Gaussian models), which is exactly what a reconstruction error or a negative log-likelihood scores directly. The One-Class SVM, by enclosing the bulk with an RBF boundary, and the IF, by isolating points with axis-aligned random splits, capture the global shape of the data well (hence their high ROC-AUC) but rank the rare positives less accurately (hence their low PR-AUC). The IF suffers most: axis-aligned splits over decorrelated PCA features isolate few of the true anomalies that sit close to the legitimate bulk, and under temporal drift it collapses almost to the prevalence floor (0.0384).

5.3 The ROC–PR gap: an empirical demonstration

All four detectors achieve a ROC-AUC clustered around 0.95, visually indistinguishable, yet their PR-AUC ranges from 0.12 to 0.63. This is a direct, data-anchored confirmation of Davis and Goadrich [2006]: under a prevalence of $\pi \approx 0.0017$ the negative class dominates the false-positive-rate denominator, so even thousands of false positives barely move the ROC curve, leaving ROC-AUC optimistically high; precision, which places those false positives next to the true positives, exposes the difference immediately. The contrast is visible already in Figure 1, where the same AE scores yield a near-perfect ROC curve but a markedly lower PR curve. Crucially, we observe the gap *twice*: across models on a fixed split (this section) and across splits for a fixed model (Section 5.4). Both observations justify our decision to treat PR-AUC as the primary metric and to report ROC-AUC only for reference.

5.4 Non-stationarity under temporal validation

Moving from the stratified to the temporal protocol roughly halves PR-AUC for every model (the AE from 0.6283 to 0.2381, the Gaussian from 0.4993 to 0.2323), and the realized fraud rate itself falls from 0.167% to 0.122%, indicating that fraud is concentrated earlier in the 48-hour window. Because the temporal split trains on the past and predicts the future, this decay is direct evidence of concept drift: the distribution of legitimate behaviour shifts over time, so a detector fitted on early data flags normal later transactions as anomalous. The deployment-realistic temporal numbers should therefore be read as the actual ceiling, with the stratified numbers as an optimistic upper bound. Tellingly, ROC-AUC barely reacts to this drift (remaining ≈ 0.92 – 0.95), so it would have declared the temporal setting essentially as healthy as the stratified one; a second manifestation of the ROC–PR gap. Figure 2 shows this for the AE: the ROC curve stays excellent while the PR curve degrades. Under drift the AE and the Gaussian become statistically tied at the top (0.2381 vs. 0.2323), so no method can be declared best in the realistic regime.

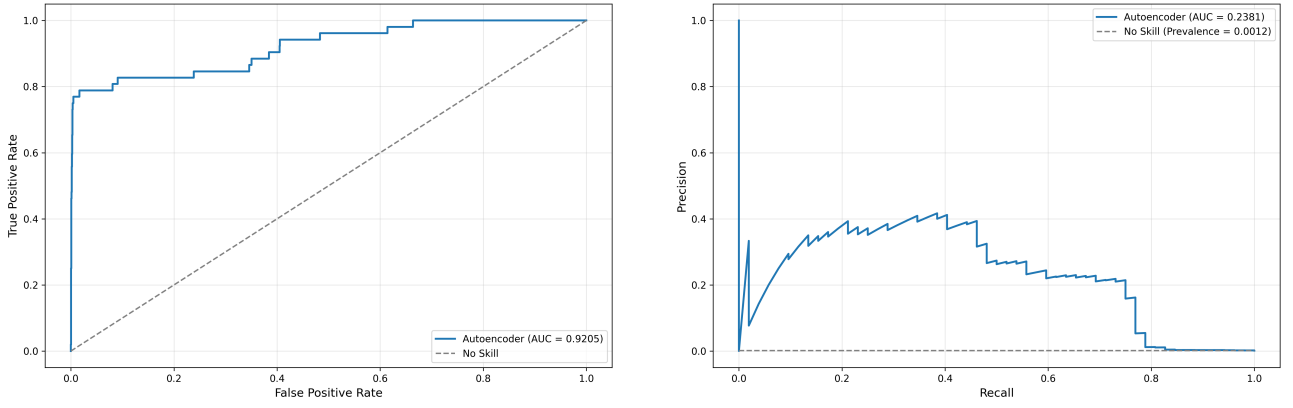


Figure 2: Autoencoder evaluation curves under the *temporal* split. The ROC curve remains near-excellent while the Precision–Recall curve degrades markedly, illustrating both concept drift and the ROC–PR gap on the same scores.

5.5 Computational cost as a deployment insight

Training cost varies by four orders of magnitude across the methods, and it does not track accuracy. The One-Class SVM takes roughly 46 minutes to fit on the $\approx 199\text{k}$ legitimate training transactions (consistent with the $O(n^2)$ – $O(n^3)$ scaling of a kernel SVM) yet ranks only third on PR-AUC. The AE costs about twenty minutes on CPU (a figure that a GPU or a larger batch size would cut substantially), while the Gaussian and the IF are essentially free (0.1 s and 0.5 s). The Gaussian’s combination of negligible training cost and a PR-AUC within the AE’s confidence interval makes it a compelling first-deployment baseline. “Cost” then enters the analysis a second time, through the asymmetric misclassification cost of Section 4.4: the sensitivity sweep shows that whether *any* detector is worth deploying depends on the business ratio C_{FN}/C_{FP} , with break-even points around 2:1 (stratified) and between 5:1 and 50:1 (temporal). Both the training cost and the operating cost, therefore, are first-class deployment considerations rather than afterthoughts.

5.6 Limitations

Several limitations qualify these conclusions. First, the results come from a single training run with a fixed seed; the bootstrap confidence intervals capture the sampling uncertainty of the test evaluation but not the stochasticity of training, and a multi-seed study was infeasible given the One-Class SVM’s fit time. Second, the test sets contain few positives (71 frauds under the stratified split, 52 under the temporal one), which makes the intervals wide and leaves close comparisons (notably the AE vs. the Gaussian) statistically unresolved; a paired bootstrap of the score difference would be a more powerful test. Third, our methods are deliberately unsupervised: supervised classifiers occupy a different point in the design space because they exploit the fraud labels that our one-class setting forgoes, so their performance is not directly comparable to ours and we do not benchmark against them here. Fourth, the cost-matrix values are illustrative rather than institution-specific; we mitigate this with the sensitivity sweep, but a real deployment would calibrate C_{FN} and C_{FP} to its own economics. Finally, features V1–V28 are anonymized PCA components, which limits feature-level interpretability and the transferability of these findings, and the dataset spans only ≈ 48 hours, so longer horizons could exhibit stronger drift than we observe.

6 Conclusion

We studied unsupervised anomaly detection for credit-card fraud under extreme class imbalance, training an AE on legitimate transactions only and using its reconstruction error as an anomaly score, benchmarked against three unsupervised baselines (Isolation Forest, One-Class SVM, and a Gaussian density model). Throughout, we treated PR-AUC as the primary metric, quantified uncertainty with bootstrap confidence intervals, compared two splitting protocols, and translated the scores into deployment-oriented operating points via an asymmetric cost matrix.

Four conclusions stand out. First, the AE achieves the best PR-AUC on the stratified split (0.6283), but its advantage over a single multivariate Gaussian (0.4993) is not statistically conclusive, and the Gaussian is essentially free to fit; a strong, cheap baseline that any deep model should be made to justify itself against. Second, there is a robust hierarchy: the AE and the Gaussian clearly outperform the boundary and tree-based detectors on PR-AUC in both splits. Third, ROC-AUC is uninformative under this prevalence (all models

score ≈ 0.95 regardless of their very different PR-AUC) empirically reproducing Davis and Goadrich [2006] both across models and across splits. Fourth, the temporal protocol roughly halves every model’s PR-AUC, exposing concept drift that ROC-AUC entirely hides, and the cost analysis shows that whether a detector is worth deploying at all depends on the business cost ratio rather than on PR-AUC alone.

In practice, we would recommend the Gaussian density model as a near-free first line of defence, reserving the AE for settings where its modest accuracy edge justifies the training cost, and we would insist on temporal validation and a calibrated cost matrix before any deployment. Future work includes multi-seed and paired-bootstrap testing to resolve the AE-vs-Gaussian comparison, richer autoencoder variants (denoising or variational), explicit drift adaptation through periodic or online retraining, and a comparison against supervised and semi-supervised methods that exploit the labels our setting deliberately forgoes.

7 Acknowledgment of AI assistance

In the preparation of this report and codebase, we used a large language model (Claude, Anthropic) as an assistant for tasks including sanity-checking mathematical derivations against the course material, prototyping code, and discussing implementation choices. All theoretical content, code, experimental results, and their interpretation were ultimately produced and reviewed by the authors. The LLM was not used to generate experimental results. Anthropic [2026]

References

- Anthropic. Claude (Opus 4.8). Large language model, <https://www.anthropic.com/claude>, 2026. Accessed June 2026.
- Jesse Davis and Mark Goadrich. The relationship between precision-recall and roc curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240. ACM, 2006.
- Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD, 4th edition, 2013.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016. <http://www.deeplearningbook.org>.
- Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining (ICDM)*, pages 413–422. IEEE, 2008.
- Michael A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015. <http://neuralnetworksanddeeplearning.com>.
- F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Bernhard Schölkopf and Alexander J. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, Cambridge, MA, 2002.
- Bernhard Schölkopf, John C. Platt, John Shawe-Taylor, Alex J. Smola, and Robert C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7):1443–1471, 2001.